

Chương 2 đã xác định đầy đủ các yêu cầu chức năng và phi chức năng của hệ thống giám sát chất lượng không khí, bao gồm thu thập dữ liệu đa thông số từ cảm biến chi phí thấp, truyền dữ liệu tầm xa qua LoRa, xử lý dữ liệu chuỗi thời gian, hiển thị dashboard thời gian thực và quản trị hệ thống. Trên cơ sở đó, Chương 3 trình bày nền tảng lý thuyết và các công nghệ được lựa chọn để hiện thực hóa hệ thống. Nội dung chương bao gồm: (i) kiến trúc tổng thể trong phần 0.1, (ii) nền tảng phần cứng Internet vạn vật (Internet of Things – IoT) trong phần 0.2, (iii) giao thức truyền thông Long Range (LoRa) trong phần 0.3, (iv) công nghệ phần mềm phía máy chủ trong phần 0.4, (v) công nghệ phần mềm phía giao diện trong phần 0.5, (vi) lý thuyết tính toán chỉ số chất lượng không khí trong phần 0.6, và (vii) hạ tầng triển khai trong phần 0.7.

0.1 Kiến trúc tổng thể hệ thống

Hệ thống được thiết kế theo kiến trúc ba tầng, bao gồm tầng cảm biến, tầng trung gian và tầng ứng dụng. Tầng cảm biến gồm các Sensor Node, mỗi node tích hợp vi điều khiển ESP32, các module cảm biến môi trường và module LoRa. Tầng trung gian là Gateway, đóng vai trò cầu nối giữa mạng LoRa và mạng Internet thông qua kết nối WiFi. Tầng ứng dụng bao gồm Backend Server xử lý nghiệp vụ và Frontend cung cấp giao diện người dùng.

Luồng dữ liệu của hệ thống diễn ra như sau. Sensor Node đọc dữ liệu từ các cảm biến, đóng gói thành gói tin nhị phân 18 byte và phát qua LoRa. Gateway nhận gói tin, lưu vào bộ đệm vòng và gửi lên Backend Server dưới dạng HTTP POST (JSON batch). Backend Server lưu dữ liệu vào cơ sở dữ liệu TimescaleDB, tính chỉ số Chỉ số chất lượng không khí (Air Quality Index – AQI), kiểm tra ngưỡng cảnh báo và phát sóng đến giao diện web thông qua Socket.IO. Kiến trúc này đảm bảo tính module hóa: mỗi tầng có thể phát triển và mở rộng độc lập mà không ảnh hưởng đến các tầng còn lại.

0.2 Nền tảng phần cứng IoT

0.2.1 Vi điều khiển ESP32

ESP32 là dòng vi điều khiển do Espressif Systems phát triển, tích hợp bộ xử lý hai nhân Xtensa LX6 32-bit hoạt động ở tần số tối đa 240 MHz, bộ nhớ SRAM 520 KB, WiFi 802.11 b/g/n và Bluetooth 4.2 **espressif2024**. Vi điều khiển này hỗ trợ đầy đủ các giao tiếp ngoại vi cần thiết cho hệ thống: UART để giao tiếp với module LoRa và cảm biến PMS7003, và I2C để giao tiếp với cảm biến CCS811, AHT10 và màn hình OLED.

Đồ án sử dụng board ESP32 DevKit V1 với framework Arduino trên nền tảng PlatformIO. Firmware của Sensor Node sử dụng FreeRTOS — Hệ điều hành thời

gian thực (Real-Time Operating System – RTOS) tích hợp sẵn trong ESP32 — để quản lý đồng thời các tác vụ đọc cảm biến, gửi dữ liệu LoRa và hiển thị OLED. Firmware của Gateway sử dụng kiến trúc superloop do chỉ cần thực hiện hai tác vụ tuần tự: nhận gói tin LoRa và chuyển tiếp lên Backend Server qua WiFi.

So với các lựa chọn thay thế, Arduino Mega không tích hợp WiFi nên không thể thực hiện provisioning qua web. STM32 có hiệu năng tương đương nhưng thiếu WiFi tích hợp và hệ sinh thái thư viện cảm biến ít phong phú hơn. Raspberry Pi có năng lực xử lý vượt trội nhưng chi phí cao gấp ba đến năm lần và tiêu thụ năng lượng lớn, không phù hợp cho node cảm biến hoạt động bằng pin.

0.2.2 Module cảm biến

Hệ thống sử dụng ba loại cảm biến để đo sáu thông số môi trường theo yêu cầu đặt ra ở Chương 2. Tất cả các cảm biến đều giao tiếp qua giao tiếp nối tiếp (UART hoặc I2C), giúp đơn giản hóa thiết kế phần cứng.

PMS7003 (Plantower) là cảm biến laser đo nồng độ bụi mịn PM2.5 và PM10 trong không khí **plantower2023**. Cảm biến hoạt động theo nguyên lý tán xạ laser: luồng không khí được hút qua buồng đo, các hạt bụi đi qua chùm laser sẽ tạo ra tín hiệu tán xạ, từ đó vi xử lý tích hợp tính toán nồng độ theo đơn vị $\mu g/m^3$. PMS7003 giao tiếp qua UART với tốc độ 9600 baud, kích thước nhỏ gọn ($48 \times 37 \times 12$ mm) và chi phí khoảng 350.000 VNĐ. Lựa chọn thay thế bao gồm PMS5003 cùng hãng Plantower với thông số tương tự nhưng tỉ lệ hỏng cao hơn, và SDS011 (Nova Fitness) có kích thước lớn hơn đáng kể ($71 \times 70 \times 23$ mm).

CCS811 (ams AG) là cảm biến chất lượng không khí sử dụng công nghệ oxit kim loại (MOX), đo đồng thời nồng độ CO₂ tương đương (eCO₂, phạm vi 400–8192 ppm) và tổng hợp chất hữu cơ bay hơi (Tổng các hợp chất hữu cơ bay hơi (Total Volatile Organic Compounds – TVOC), phạm vi 0–1187 ppb) **ams2023**. Cảm biến giao tiếp qua I2C, chi phí khoảng 150.000 VNĐ. Lựa chọn thay thế bao gồm SGP30 (Sensirion) có thông số tương đương và SCD30 (Sensirion) sử dụng công nghệ NDIR cho độ chính xác cao hơn nhưng chi phí đắt gấp năm lần, vượt mục tiêu chi phí thấp của đề tài.

AHT10 (Aosong Electronics) là cảm biến đo nhiệt độ (phạm vi -40 đến $85^{\circ}C$, sai số $\pm 0.3^{\circ}C$) và độ ẩm tương đối (phạm vi 0–100% RH, sai số $\pm 2\%$ RH) **aosong2023**. Cảm biến sử dụng giao tiếp I2C (địa chỉ 0x38), chia sẻ chung bus I2C với CCS811, giúp giảm số chân GPIO cần sử dụng trên ESP32. Chi phí rất thấp (khoảng 35.000 VNĐ). Lựa chọn thay thế gồm DHT22 (cùng hãng Aosong, giao tiếp 1-wire, sai số $\pm 0.5^{\circ}C$) và SHT31 (Sensirion) có độ chính xác cao hơn ($\pm 0.2^{\circ}C$) nhưng chi phí gấp năm lần.

Bảng 1 tổng hợp thông số kỹ thuật của các cảm biến được sử dụng trong hệ thống.

Bảng 1: Thông số kỹ thuật các cảm biến sử dụng trong hệ thống

Cảm biến	Thông số đo	Phạm vi	Giao tiếp	Chi phí
PMS7003	PM2.5, PM10	0–500 $\mu g/m^3$	UART	~10 USD
CCS811	eCO ₂ , eTVOC	400–8192 ppm, 0–1187 ppb	I2C	~8 USD
AHT10	Nhiệt độ, độ ẩm	–40–85°C, 0–100% RH	I2C	~2 USD

0.3 Giao thức truyền thông LoRa

0.3.1 Tổng quan về LoRa

LoRa (Long Range) là kỹ thuật điều chế lớp vật lý do Semtech phát triển, sử dụng phương pháp trải phổ Chirp Spread Spectrum (CSS) trên băng tần sub-GHz không cần cấp phép (433 MHz, 868 MHz hoặc 915 MHz tùy khu vực) **augustin2016**. Đặc điểm nổi bật của LoRa là khả năng truyền tầm xa (5–15 km trong điều kiện tầm nhìn thẳng) với mức tiêu thụ năng lượng rất thấp, tuy nhiên tốc độ truyền dữ liệu bị giới hạn (0,3–37,5 kbps). Đặc tính này phù hợp với bài toán giám sát môi trường, nơi khối lượng dữ liệu nhỏ (18 byte mỗi gói tin) nhưng yêu cầu phạm vi phủ sóng rộng.

Cần phân biệt LoRa (lớp vật lý) với LoRaWAN (giao thức mạng). LoRaWAN định nghĩa kiến trúc mạng và giao thức truyền thông trên nền LoRa, bao gồm quản lý thiết bị, mã hóa và kiến trúc hình sao. Đồ án sử dụng LoRa ở chế độ point-to-point (không qua LoRaWAN) để đơn giản hóa kiến trúc và giảm chi phí, vì hệ thống chỉ cần truyền dữ liệu từ Sensor Node đến Gateway trong phạm vi một khu vực (campus hoặc khu đô thị).

Kết quả khảo sát ở Chương 2 cho thấy tất cả bốn hệ thống hiện tại (PAM Air, IQAir, PurpleAir, VN-AQI) đều sử dụng WiFi hoặc 4G, yêu cầu hạ tầng mạng tại mỗi điểm đo. LoRa giải quyết hạn chế này bằng cách cho phép truyền dữ liệu tầm xa mà không cần WiFi tại mỗi node. Các lựa chọn thay thế bao gồm: (i) WiFi có tầm phủ ngắn (dưới 100 m) và cần điểm truy cập tại mỗi vị trí, (ii) Zigbee có tầm phủ ngắn (dưới 100 m) dù tiêu thụ năng lượng thấp, (iii) NB-IoT và 4G yêu cầu SIM và phí dữ liệu hàng tháng, và (iv) Sigfox phụ thuộc vào nhà mạng Sigfox chưa phủ sóng tại Việt Nam.

0.3.2 Module AS32-TTL-100

Đồ án sử dụng module AS32-TTL-100 dựa trên chip SX1278 (Semtech), hoạt động ở tần số 433 MHz ISM band với công suất phát 100 mW (20 dBm). Module giao tiếp với vi điều khiển qua UART ở chế độ transparent — nghĩa là dữ liệu ghi vào cổng UART sẽ được module tự động điều chế và phát qua sóng LoRa, và ngược lại. Chế độ này đơn giản hóa đáng kể firmware so với việc sử dụng chip SX1278 trực tiếp qua SPI, vốn yêu cầu cấu hình thành ghi phức tạp.

Module hỗ trợ bốn chế độ hoạt động được điều khiển qua hai chân MD0 và MD1: chế độ truyền bình thường (MD0=0, MD1=0), chế độ tiết kiệm năng lượng, chế độ cấu hình và chế độ ngủ (MD0=1, MD1=1). Khoảng cách truyền lý thuyết đạt 3–5 km trong môi trường đô thị và trên 10 km trong điều kiện tầm nhìn thẳng.

0.4 Công nghệ phần mềm phía máy chủ

0.4.1 Node.js và Express

Node.js là môi trường thực thi JavaScript phía máy chủ, xây dựng trên engine V8 của Google Chrome **nodejs2024**. Đặc điểm quan trọng nhất của Node.js là mô hình event-driven, non-blocking I/O: thay vì tạo luồng mới cho mỗi kết nối (như mô hình truyền thống), Node.js sử dụng vòng lặp sự kiện đơn luồng để xử lý đồng thời hàng nghìn kết nối. Mô hình này phù hợp với hệ thống giám sát thời gian thực, nơi Backend cần duy trì nhiều kết nối Socket.IO đồng thời và xử lý dữ liệu telemetry liên tục từ các Gateway.

Express là framework web phổ biến nhất cho Node.js, cung cấp hệ thống middleware, routing và xử lý HTTP request/response. Đồ án sử dụng Express để xây dựng REST API cho các chức năng CRUD thiết bị, truy vấn dữ liệu, xuất CSV và quản lý tài khoản. Các middleware bảo mật bao gồm Helmet (thiết lập HTTP security headers), CORS (kiểm soát truy cập cross-origin) và xác thực JWT.

So với các lựa chọn thay thế: Python kết hợp Flask hoặc FastAPI có hiệu năng hạn chế hơn cho kịch bản real-time do mô hình đồng thời khác biệt. Java kết hợp Spring Boot có kiến trúc vững chắc nhưng chi phí tài nguyên và độ phức tạp cao cho dự án quy mô nhỏ. Go kết hợp Gin có hiệu năng vượt trội nhưng hệ sinh thái thư viện nhỏ hơn đáng kể.

0.4.2 PostgreSQL, TimescaleDB và PostGIS

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, tuân thủ ACID và hỗ trợ hệ thống extension phong phú **postgresql2024**. Đồ án mở rộng PostgreSQL bằng hai extension chuyên biệt: TimescaleDB và PostGIS.

TimescaleDB là extension chuyển đổi PostgreSQL thành cơ sở dữ liệu chuỗi

thời gian (time-series), được thiết kế cho dữ liệu có nhãn thời gian liên tục **timescaledb2024**. Đồ án sử dụng ba tính năng chính của TimescaleDB. Thứ nhất, *Hypertable*: TimescaleDB tự động phân mảnh (partition) bảng dữ liệu theo thời gian, giúp tối ưu hiệu năng ghi và truy vấn khi lượng dữ liệu tăng liên tục. Thứ hai, *Continuous Aggregate*: materialized view được tự động cập nhật khi có dữ liệu mới, cho phép truy vấn lịch sử tổng hợp với hiệu năng cao mà không cần tính toán lại từ dữ liệu thô — đáp ứng yêu cầu phi chức năng truy vấn 30 ngày trong vòng 3 giây (Bảng ??). Thứ ba, *Retention Policy*: tự động xóa dữ liệu cũ theo chu kỳ cấu hình để kiểm soát dung lượng lưu trữ.

PostGIS là extension bổ sung kiểu dữ liệu không gian và các hàm xử lý địa lý cho PostgreSQL **postgis2024**. PostGIS cho phép lưu trữ tọa độ GPS dưới dạng đối tượng hình học và thực hiện các phép tính khoảng cách địa lý, từ đó hỗ trợ truy vấn trạm đo gần nhất dựa trên vị trí người dùng — đáp ứng yêu cầu chức năng tự động gợi ý trạm gần nhất trên dashboard (Bảng ??). Chi tiết cách áp dụng TimescaleDB và PostGIS vào thiết kế cơ sở dữ liệu sẽ được trình bày trong Chương 4.

So với các lựa chọn thay thế: InfluxDB chuyên biệt cho time-series nhưng không hỗ trợ dữ liệu không gian, buộc phải sử dụng hai cơ sở dữ liệu riêng biệt. MongoDB là cơ sở dữ liệu NoSQL linh hoạt nhưng không tối ưu cho truy vấn time-series aggregate. MySQL thiếu extension tương đương TimescaleDB và PostGIS.

0.4.3 Prisma ORM

Prisma là ORM thế hệ mới cho Node.js, áp dụng phương pháp schema-first: lược đồ cơ sở dữ liệu được định nghĩa trong một file khai báo, từ đó Prisma Client tự động sinh ra các hàm truy vấn type-safe **prisma2024**.

Ưu điểm chính của Prisma so với viết SQL thủ công là giảm lỗi runtime nhờ kiểm tra kiểu tại thời điểm biên dịch, hỗ trợ migration tự động khi thay đổi lược đồ, và cú pháp truy vấn trực quan. Ngoài ra, Prisma cho phép kết hợp với raw SQL khi cần sử dụng các tính năng đặc thù của extension (TimescaleDB, PostGIS) mà ORM chưa hỗ trợ trực tiếp.

0.4.4 Socket.IO

Socket.IO là thư viện giao tiếp thời gian thực hai chiều giữa máy chủ và trình duyệt, xây dựng trên nền WebSocket với cơ chế fallback về HTTP long-polling khi WebSocket không khả dụng **socketio2024**. Đồ án sử dụng Socket.IO cho hai chức năng: (i) phát sóng (broadcast) dữ liệu đo mới đến tất cả giao diện dashboard khi Backend nhận dữ liệu từ Gateway, và (ii) phát thông báo cảnh báo thời gian thực đến giao diện quản trị khi phát hiện giá trị vượt ngưỡng.

So với WebSocket thuần, Socket.IO cung cấp thêm cơ chế tự động kết nối lại (auto-reconnect), hệ thống phòng (room) để phát sóng có chọn lọc và xử lý graceful degradation. So với Server-Sent Events (SSE), Socket.IO hỗ trợ giao tiếp hai chiều, cần thiết cho các tính năng tương tác trong tương lai.

0.5 Công nghệ phần mềm phía giao diện

0.5.1 React và Vite

React là thư viện xây dựng giao diện người dùng do Meta phát triển, dựa trên kiến trúc component — mỗi thành phần giao diện là một đơn vị độc lập, có thể tái sử dụng và kết hợp để tạo nên giao diện phức tạp **react2024**. React sử dụng Virtual DOM để tối ưu hiệu năng cập nhật giao diện: khi trạng thái thay đổi, React so sánh Virtual DOM mới với phiên bản trước, chỉ cập nhật các phần tử thực sự thay đổi trên DOM thật. Cơ chế này phù hợp với dashboard thời gian thực, nơi dữ liệu liên tục được cập nhật qua Socket.IO nhưng chỉ một số thành phần giao diện cần vẽ lại.

Vite là công cụ build thế hệ mới, sử dụng ES Module trên trình duyệt trong quá trình phát triển và Rollup để đóng gói sản phẩm **vite2024**. Vite cung cấp Hot Module Replacement (HMR) gần như tức thì, tăng tốc đáng kể quy trình phát triển so với Webpack truyền thống.

So với các lựa chọn thay thế: Vue.js nhẹ hơn nhưng hệ sinh thái thư viện bên thứ ba nhỏ hơn. Angular là framework toàn diện nhưng có đường cong học tập dốc và chi phí tài nguyên cao cho dự án quy mô nhỏ. Svelte có hiệu năng runtime tốt nhưng hệ sinh thái chưa trưởng thành.

0.5.2 Thư viện trực quan hóa dữ liệu

Đồ án sử dụng hai thư viện trực quan hóa chính. **Leaflet** là thư viện bản đồ tương tác mã nguồn mở, nhẹ (khoảng 42 KB gzip) và hỗ trợ đầy đủ các tính năng bản đồ: marker, popup, zoom và lớp phủ **leaflet2024**. Đồ án sử dụng Leaflet kết hợp React-Leaflet để hiển thị bản đồ AQI với marker mã màu theo chuẩn Cơ quan Bảo vệ Môi trường Hoa Kỳ (United States Environmental Protection Agency – US EPA) tại mỗi trạm đo. So với Google Maps API (phải trả phí sau số lượng request nhất định) và Mapbox (freemium, yêu cầu API key), Leaflet hoàn toàn miễn phí và sử dụng tile từ OpenStreetMap.

Apache ECharts là thư viện biểu đồ hiệu năng cao, hỗ trợ hơn 20 loại biểu đồ và khả năng xử lý tập dữ liệu lớn **echarts2024**. Đồ án sử dụng ECharts thông qua wrapper `echarts-for-react` để hiển thị biểu đồ lịch sử dữ liệu theo giờ và theo ngày. So với Chart.js (nhẹ hơn nhưng ít tính năng nâng cao), ECharts hỗ trợ tốt hơn cho responsive chart, large dataset rendering và tooltip tương tác phức tạp.

0.5.3 Quản lý trạng thái và đa ngôn ngữ

Đồ án sử dụng Zustand làm thư viện quản lý trạng thái toàn cục cho ứng dụng React. Zustand có thiết kế tối giản, không yêu cầu boilerplate code và sử dụng hook-based API tương thích tự nhiên với React functional component. So với Redux (tiêu chuẩn công nghiệp nhưng yêu cầu nhiều boilerplate: action, reducer, middleware) và React Context (tích hợp sẵn nhưng gây re-render không cần thiết khi state lớn), Zustand cân bằng giữa sự đơn giản và khả năng mở rộng.

Đồ án hỗ trợ đa ngôn ngữ (Tiếng Việt và Tiếng Anh) thông qua i18next — framework quốc tế hóa phổ biến cho JavaScript, kết hợp với react-i18next để tích hợp vào React component. Tính năng này đáp ứng yêu cầu phi chức năng về tính dễ sử dụng nêu trong Bảng ??.

0.6 Lý thuyết tính toán chỉ số chất lượng không khí

Chỉ số chất lượng không khí (AQI) là thước đo tổng hợp mức độ ô nhiễm không khí, được US EPA định nghĩa trên thang 0–500 với sáu mức: Tốt (0–50), Trung bình (51–100), Không tốt cho nhóm nhạy cảm (101–150), Có hại (151–200), Rất có hại (201–300) và Nguy hiểm (301–500) **usepa2018**. Đồ án tính AQI cho từng thông số (PM2.5, PM10) theo công thức nội suy tuyến tính của US EPA:

$$I_p = \frac{I_{Hi} - I_{Lo}}{BP_{Hi} - BP_{Lo}} \times (C_p - BP_{Lo}) + I_{Lo} \quad (1)$$

Trong đó I_p là giá trị AQI của chất ô nhiễm p , C_p là nồng độ đo được, BP_{Hi} và BP_{Lo} là các điểm ngắt nồng độ (breakpoint) trên và dưới chứa C_p , I_{Hi} và I_{Lo} là giá trị AQI tương ứng với các điểm ngắt đó. Chỉ số AQI tổng hợp là giá trị lớn nhất trong các AQI thành phần. Các điểm ngắt được tra cứu từ bảng tiêu chuẩn của US EPA **usepa2018**. Hệ thống áp dụng công thức này trên Backend Server mỗi khi nhận dữ liệu mới từ Gateway, đồng thời ánh xạ kết quả sang mã màu tương ứng để hiển thị trên bản đồ và dashboard.

0.7 Hạ tầng triển khai

0.7.1 Docker và Docker Compose

Docker là nền tảng container hóa cho phép đóng gói ứng dụng cùng toàn bộ phụ thuộc (thư viện, runtime, cấu hình) vào một đơn vị triển khai nhất quán gọi là container **docker2024**. Docker Compose mở rộng Docker cho ứng dụng đa container, cho phép định nghĩa và quản lý toàn bộ hệ thống trong một file `docker-compose.yml`.

Đồ án container hóa bốn thành phần: (i) cơ sở dữ liệu PostgreSQL/TimescaleDB,

(ii) Backend Server (Node.js), (iii) Nginx phục vụ static files và reverse proxy, và (iv) Certbot quản lý chứng chỉ SSL. Việc container hóa đảm bảo môi trường nhất quán giữa phát triển và sản xuất, đơn giản hóa quy trình triển khai trên VPS chỉ với một lệnh duy nhất. So với triển khai trực tiếp trên VPS (cài đặt phụ thuộc thủ công, khó tái tạo), Docker loại bỏ hoàn toàn vấn đề “hoạt động trên máy em nhưng không hoạt động trên server”. So với Kubernetes, Docker Compose đơn giản và phù hợp hơn cho hệ thống quy mô nhỏ chạy trên một máy chủ duy nhất.

0.7.2 Nginx và Certbot

Nginx đóng vai trò reverse proxy, chuyển tiếp các yêu cầu HTTP/WebSocket từ client đến Backend Server, đồng thời phục vụ các tệp tĩnh (static files) của Frontend đã được build bởi Vite. Kiến trúc này giúp Backend Server không bị lộ trực tiếp ra Internet, tăng cường bảo mật và cho phép cấu hình HTTPS tập trung tại Nginx.

Certbot là công cụ tự động hóa việc cấp phát và gia hạn chứng chỉ SSL/TLS miễn phí từ Let's Encrypt. Đồ án cấu hình Certbot chạy như một container riêng, tự động gia hạn chứng chỉ mỗi 12 giờ, đảm bảo giao diện web luôn được truy cập qua HTTPS — đáp ứng yêu cầu phi chức năng về bảo mật (Bảng ??).

Chương này đã trình bày nền tảng lý thuyết và các công nghệ được lựa chọn cho từng tầng trong kiến trúc hệ thống. Ở tầng cảm biến, vi điều khiển ESP32 kết hợp ba module cảm biến (PMS7003, CCS811, AHT10) cho phép đo sáu thông số môi trường với chi phí thấp. Giao thức LoRa qua module AS32-TTL-100 giải quyết bài toán truyền dữ liệu tầm xa mà không cần hạ tầng WiFi tại mỗi điểm đo. Ở tầng máy chủ, Node.js kết hợp TimescaleDB và Socket.IO đáp ứng yêu cầu xử lý dữ liệu chuỗi thời gian và cập nhật thời gian thực. Ở tầng giao diện, React kết hợp Leaflet và ECharts cung cấp dashboard trực quan. Toàn bộ hệ thống được container hóa bằng Docker để đơn giản hóa triển khai. Với các công nghệ đã lựa chọn, Chương 4 sẽ trình bày chi tiết thiết kế kiến trúc, triển khai các chức năng và đánh giá kết quả thực nghiệm.